Company: Google

Interview Type: DSA

Q1: Optimizing Data Pipeline Throughput with Cold-Start Penalties

Difficulty: Hard

Tags: Dynamic Programming, Greedy, String

Problem Statement: You are given a sequence of tasks of different types (e.g., ['A', 'B', 'A', 'C']). Executing a task takes 1 unit of time. However, if a task is of a different type than the immediately preceding task, a "cold-start" penalty of K units of time is added. You are allowed to reorder the tasks to minimize the total completion time, but you must maintain the relative order of tasks belonging to the same type (i.e., the i-th 'A' must still be processed before the $(i+1)$-th 'A').

Expected Approach: Use a greedy approach with a priority queue to always pick the task type that has the most remaining occurrences to reduce the frequency of switches.

Optimized Approach: Recognize that to minimize penalties, you must minimize the number of transitions. This is equivalent to grouping all tasks of the same type together. The total time will be N (total tasks) + $(M-1) \times K$, where M is the number of unique task types present. If the problem adds a constraint that no two tasks of the same type can be within D distance, it becomes a scheduling problem using a Max-Heap to track frequencies and a cooling queue for task availability.

Time Complexity: O(N log M) where N is the number of tasks and M is the number of unique types.

Space Complexity: O(M) to store frequencies.

Optimizations: Use an array for frequency counting if the character set is fixed (e.g., ASCII).

Hints: What is the primary factor increasing the total time? How can we minimize the "switches" between different types?

Q2: Detecting Ancestral Conflicts in a Distributed Version Control Tree

Difficulty: Medium-Hard

Tags: Tree, Lowest Common Ancestor (LCA), Euler Tour

Problem Statement: In a version control system, the history is represented as a tree where each node is a commit. A conflict occurs if two users branch from a common ancestor and modify the same file. Given a list of M pairs of nodes representing concurrent edits, and a target file's "original" version node V, determine for each pair if they share V as their most recent common ancestor.

Expected Approach: For each pair (u, v), calculate the Lowest Common Ancestor (LCA). Compare if $LCA(u, v) == V$.

Optimized Approach: Preprocess the tree using an Euler Tour and a Segment Tree (or Sparse Table) for RMQ (Range Minimum Query) to answer LCA queries in $O(1)$. Alternatively, use Binary Lifting. To handle the "most recent" part, check if V is an ancestor of both u and v using entry/exit timestamps, and ensure no other node V' that is a descendant of V is also an ancestor of both.

Time Complexity: O(N log N) preprocessing, O(1) or O(log N) per query.

Space Complexity: O(N log N) for the jump table or O(N) for Euler Tour.

Optimizations: Use the 1-bit space optimization in Binary Lifting or a Tarjan's offline LCA if all pairs are known in advance.

Hints: How can you quickly determine if node A is an ancestor of node B? (Think entry/exit times). If V is the LCA, it means it is the nearest point where the paths from u and v to the root meet.

Q3: Grid Traversal with Mandatory Recharge Stations

Difficulty: Hard

Tags: Dijkstra, Binary Search, State-Space Search

Problem Statement: You are given an $N \times M$ grid where each cell (r, c) has an entry cost $E_{r,c}$ representing energy consumed. You start at $(0,0)$ with a battery of capacity B and must reach $(N-1, M-1)$. Certain cells are "Recharge Stations" that instantly refill your battery to B upon entry. If your current energy is less than $E_{r,c}$, you cannot enter the cell. Find the minimum battery capacity B required to complete the journey.

Expected Approach: Binary search over the possible range of B $[0, \sum E_{r,c}]$. For a fixed B, use BFS or Dijkstra to check if the destination is reachable.

Optimized Approach: Use a modified Dijkstra where the state is (r, c) and the value stored is the maximum remaining energy at that cell. Initialize `rem_energy[0][0] = B - cost[0][0]`. For each neighbor, `new_energy = current_energy - cost[neighbor]`. If the neighbor is a station, `new_energy = B - cost[neighbor]`. Use a Max-Priority Queue to always expand the path with the most remaining energy.

Time Complexity: $O(NM \log(NM) \cdot \log(\text{Max\_Energy}))$

Space Complexity: $O(NM)$

Optimizations: Use a 1D array for Dijkstra if memory is constrained; stop early if the target is reached with a non-negative energy value.

Hints: Think of this as a "shortest path" problem where you maximize the "minimum energy" remaining. The battery reset at stations makes standard Dijkstra tricky without the correct state.

Q4: Maximum Path Sum in a Tree after Single Edge Deletion

Difficulty: Hard

Tags: Tree DP, Re-rooting DP, Graph

Problem Statement: Given a tree with N nodes, each assigned an integer weight (positive or negative), find the maximum possible path sum in any component formed after deleting exactly one edge from the tree. A path sum is the sum of node weights on a simple path between any two nodes in a component.

Expected Approach: For every edge, remove it and run two separate "Maximum Path Sum in a Tree" (standard DP) operations on the resulting components. This is $O(N^2)$.

Optimized Approach: Use Tree DP to calculate the max path passing through a node, the max path in its subtree, and the top-3 longest downward paths. Use Re-rooting DP (top-down pass) to calculate the maximum path sum in the "upper" component for every edge. For any edge (u, v) where v is a child of u, the answer is $\max(\text{max\_path\_in\_subtree}(v), \text{max\_path\_outside\_subtree}(v))$.

Time Complexity: $O(N)$

Space Complexity: $O(N)$

Optimizations: Pre-calculate the diameter of the tree and store downward paths in a sorted list to simplify the re-rooting logic.

Hints: Deleting an edge (u, v) splits the tree into two. One is the subtree rooted at v. The other is the rest of the tree. Can you calculate the max path in both in a single pass?

Q5: Shortest Path in a Grid with K-Obstacle Breaks and Recharge Stations

Difficulty: Hard

Tags: BFS, Dijkstra, State-Space Search

Problem Statement: Given an $M \times N$ grid where 0 is an empty cell, 1 is a wall, and 2 is a recharge station. You start at $(0,0)$ with E energy and K "wall-break" tokens. Moving to an adjacent empty cell costs 1 energy. You can move into a wall cell by consuming 1 token and 1 energy. Entering a recharge station (2) restores your energy to E (tokens are not restored). Find the minimum steps to reach $(M-1, N-1)$. If unreachable, return -1.

Expected Approach: A standard BFS will not work because the state depends on remaining energy and remaining tokens. A modified Dijkstra or BFS using a 4D state $(row, col, current\_energy, remaining\_tokens)$ is required.

Optimized Approach: Use a Priority Queue (Dijkstra) where the state is $(steps, r, c, e, k)$. To avoid redundant processing, maintain a 3D memoization table `visited[r][c][k]` storing the maximum energy seen at that position with k tokens. If a new path reaches (r, c) with k tokens but has less energy than a previously recorded state with the same or more tokens at the same location, prune that path.

Time Complexity: $O(M \cdot N \cdot K \cdot E)$

Space Complexity: $O(M \cdot N \cdot K)$

Optimizations: Use a bitmask if K and E are very small, or use a 3D array for distance to prune states where the current energy is lower than a previously visited state with the same token count.

Hints: Think of this as a shortest path on a layered graph. What defines a "visited" state uniquely? Is it just the coordinates?

Q6: Maximum Sum of Non-Adjacent Nodes in a Tree with Distance Constraint D

Difficulty: Hard

Tags: Dynamic Programming, Trees, DFS

Problem Statement: Given a tree with N nodes, each having a weight W_i, find the maximum sum of weights of a subset of nodes such that the distance between any two selected nodes is at least D.

Expected Approach: Use Tree DP. For each node, maintain a DP state representing the maximum sum considering the subtree rooted at that node, where the closest selected node in the subtree is at distance i (where $0 \le i \le D$).

Optimized Approach: Define $dp[u][i]$ as the max sum in the subtree of u, where the distance from u to the nearest selected node is i. If $i=0$, node u is selected. For each child v of u:
1. If u is selected ($i=0$), children must have their nearest selected node at distance at least $D-1$.
2. If u is not selected ($i > 0$), at least one child must provide a selected node such that the distance to u becomes i, while other children must have selected nodes at distances that satisfy the D constraint relative to that node.
Use a suffix-max array to optimize the combination of child states from $O(D^2)$ to $O(D)$.

Time Complexity: $O(N \cdot D)$

Space Complexity: $O(N \cdot D)$

Optimizations: For large D, notice that we only care about distances up to D. Use a deque or a sliding window if the tree is a path, but for a tree, standard DP with suffix maximums is most efficient.

Hints: How does the choice of a node in a subtree affect the possible choices in the parent's level? What is the minimum information needed to pass up the tree?

Q7: Minimum Max-Latency with K-Bypass Links

Difficulty: Hard

Tags: Graph, Binary Search, BFS/Dijkstra

Problem Statement: You are given a directed weighted graph with N nodes and M edges representing a network. Each edge has a latency L_i. You are allowed to select at most K edges and reduce their latency to 0 (bypassing them). Find the minimum possible value of X such that there exists a path from source node S to destination node T where all remaining (non-bypassed) edges have a latency $\le X$.

Expected Approach: Use a combination of Binary Search on the answer and a modified BFS. Binary search for the minimum possible X in the range $[0, \text{max\_latency}]$. For a fixed X, treat all edges with latency $\le X$ as weight 0 and edges with latency $> X$ as weight 1. The problem then reduces to finding if the shortest path from S to T in this 0-1 graph is $\le K$.

Optimized Approach: Instead of a full Dijkstra, use a 0-1 BFS (using a Deque) to check the condition for a fixed X in $O(N + M)$ time. The overall complexity becomes $O((N + M) \log(\text{max\_latency}))$.

Time Complexity: $O((N + M) \log(\text{max\_latency}))$

Space Complexity: $O(N + M)$

Optimizations: Use coordinate compression on the edge weights before binary searching if the range of latencies is extremely large ($> 10^9$).

Hints: 1. If you can reach T using K bypasses for a threshold X, you can also reach it for any $X' > X$. 2. The problem asks to minimize the maximum edge weight, which often suggests binary search on the result.

Q8: Longest Continuous Buffer with K-Removals

Difficulty: Hard

Tags: Sliding Window, Monotonic Queue, Greedy

Problem Statement: Given an array of N integers representing the sizes of data packets and an integer S representing the maximum buffer capacity. You can select a continuous subarray of packets, but you are allowed to "drop" (delete) up to K packets from that subarray to make the sum of the remaining packets $\le S$. Find the maximum possible length of the original subarray before deletions.

Expected Approach: Use a sliding window $[L, R]$. For each window, the goal is to keep the sum of the elements minus the sum of the K largest elements $\le S$. This requires maintaining the top K elements in the current window.

Optimized Approach: Use two pointers for the sliding window. To efficiently track the K largest elements and the sum of the rest, use two balanced BSTs or two heaps (one max-heap for the top K and one min-heap for the rest). As the window slides, move elements between heaps to maintain the property. The sum of the top K elements can be updated in $O(\log N)$ or $O(\log K)$ per element.

Time Complexity: $O(N \log K)$

Space Complexity: $O(N)$

Optimizations: A Fenwick tree or Segment tree over the sorted values of the input can also be used to find the "sum of the smallest M elements" in a range, where $M = (\text{window size} - K)$.

Hints: 1. If a window of size W is valid, is a window of size $W-1$ also valid? 2. How can you efficiently update the sum of the K largest elements when adding or removing one element from the window?

Q9: Minimum Operations to Equalize Tree Node Values

Difficulty: Hard

Tags: Tree, Greedy, DFS

Problem Statement: You are given a tree with N nodes, where each node i has an integer value V_i. In one operation, you can choose any two adjacent nodes and increase the value of one by 1 while decreasing the value of the other by 1 (total sum remains constant). Given that the sum of all V_i is divisible by N, find the minimum number of operations required to make all node values equal.

Expected Approach: Calculate the target value $T = \text{sum}(V) / N$. For any subtree rooted at node u, the net flow of "value" that must pass through the edge connecting u to its parent is the absolute difference between the current sum of the subtree and the required sum ($T \times \text{size of subtree}$).

Optimized Approach: Perform a post-order traversal (DFS). For each node u, compute `balance = V[u] - T + sum(balance of children)`. The total number of operations is the sum of `abs(balance)` for all nodes in the tree.

Time Complexity: O(N)

Space Complexity: O(H) where H is the height of the tree.

Optimizations: Use an iterative DFS to avoid stack overflow for highly skewed trees.

Hints: Think about the net flow across each edge. Does the order of operations matter for the total count?

Q10: Maximum Median Path in a Weight-Constrained Grid

Difficulty: Hard

Tags: Binary Search, Dynamic Programming, Matrix

Problem Statement: Given an $N \times M$ grid of integers, find a path from $(0,0)$ to $(N-1,N-1)$ moving only right or down such that the median of the values on the path is maximized. For a path of length L, the median is the $\lfloor (L+1)/2 \rfloor$-th smallest element.

Expected Approach: Use binary search on the possible median values (all unique values in the grid). For a fixed value X, transform the grid: elements $\ge X$ become $+1$ and elements $< X$ become -1.

Optimized Approach: After transformation, the problem becomes: "Is there a path with a sum > 0?" This can be solved using DP where $dp[i][j] = \max(dp[i-1][j], dp[i][j-1]) + \text{transformed\_value}[i][j]$. If $dp[N-1][M-1] > 0$, the median can be at least X.

Time Complexity: O(N * M * log(Number of unique elements))

Space Complexity: O(N * M) or O(M) with space optimization.

Optimizations: Pre-sort unique grid values to perform binary search efficiently. Use a bitset if the grid is small but the path is long (though not applicable here).

Hints: If X is the median, at least half the numbers in the path must be $\ge X$. How can you represent "greater than or equal to" numerically?
